

SOFTWARE DESIGN DOCUMENT

(SDD)

Moghul Travel & Tours

Website Management System

Version	1.0.0
Date	30 August 2026
Document ID	SDD-MTT-2026
Status	Draft
Companion Document	SRS-MTT-2026

Prepared by:

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) provides a comprehensive and structured system design for the Moghul Travel & Tours Website Management System (MTT System). It translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS-MTT-2026) into a detailed architectural and technical design that will guide implementation.

This document describes the system architecture, boundary/control/entity classes, data handling, and the interaction between layers, so the system can be developed consistently and maintained beyond the original developer. It establishes the design guidelines and structural decisions used to verify that the system satisfies the requirements defined in the SRS, while supporting scalability, security, and maintainability throughout the system's lifecycle. It is intended for the developer, any future developer or agency who inherits the codebase, and the business owner as a technical reference.

1.2 System Identification

System Title	Moghul Travel & Tours Website Management System
Year of Development	2026
System Identification Number	MTT-2026-V01 (MTT = Moghul Travel & Tours, 2026 = Implemented Year, V01 = Version Number)
Documentation Identification ID	SDD-MTT-2026 (SDD = Software Design Document, MTT = system, 2026 = Year Document Created)
Use Case ID	UC-MTT-001 (UC = Use Case, MTT = system, 001 = Use Case Number)
Requirement ID	REQ-MTT-001-001 (REQ = Requirement, MTT = system, 001 = Use Case Number, 001 = Requirement Number)
Package ID	PKG-MTT-001 (PKG = Package, MTT = system, 001 = Package Number)
Class ID	PKG-MTT-001-001 (PKG = Package, MTT = system, 001 = Package Number, 001 = Class Number)

Table 1.1 – System Identification

1.3 System Overview

The MTT System is organised into seven feature modules, matching the modules defined in SRS-MTT-2026, plus three cross-cutting architectural layers (Controllers, Models, and Middleware) that support them.

Module 1: User Access & Account Management allows Admin and Master Admin users to securely log in, log out, and manage staff access. The Master Admin can create new Admin accounts by invitation, and deactivate or delete accounts for staff who have left.

Module 2: Manage Packages allows Admin users to create, edit, publish, and delete Umrah, Ziarah, Group Tour, and Domestic travel packages. Visitors can browse published packages and view full package detail pages.

Module 3: Manage Inquiries allows visitors to submit inquiry forms with their contact details and package interest. Admin users view and manage these inquiries from a dashboard, and the system sends an email notification on each new submission.

Module 4: Manage Gallery allows Admin users to upload and organise trip photographs by destination, which visitors can then browse and filter on the public site.

Module 5: Manage Testimonials allows Admin users to add and manage customer reviews, which are displayed to build trust with prospective travellers.

Module 6: Manage Content Pages allows Admin users to edit static site content — the About Us story, contact details, office hours, MOTAC licensing information, and social media links — without needing a code deployment.

Module 7: Manage Announcements allows Admin users to publish time-sensitive promotions and notices, which are automatically hidden once they expire.

These seven modules are supported by three architectural layers described in Section 3: a Business Service Layer containing Controller classes (request handling and business rules) and Model classes (data structures matching the database schema), and a Middleware Layer containing the external services the system depends on (authentication, file storage, email, and CAPTCHA verification). The technology stack and architecture pattern used to implement these layers is described in Section 2.

1.4 References

SRS-MTT-2026 — Software Requirements Specification, Moghul Travel & Tours Website Management System.

Institute of Electrical and Electronics Engineers. (2018). IEEE Standard for Software Requirements Specifications (IEEE Std 29148-2018). IEEE.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Next.js Documentation — <https://nextjs.org/docs>

Supabase Documentation — <https://supabase.com/docs>

Resend Documentation — <https://resend.com/docs>

OWASP Top 10 (2021) — <https://owasp.org/Top10/>

2. ARCHITECTURE PATTERN & TECHNOLOGY STACK

2.1 Architecture Pattern

The MTT System is designed as a Modular Monolith using a Three-Tier Architecture integrated with a Backend-as-a-Service (BaaS) provider. A single Next.js codebase is deployed as one unit, internally organised by feature module, rather than split into separate microservices — appropriate for a small business's operational capacity while keeping each module's boundary, control, and data concerns cleanly separated.

Tier	Technology	Responsibility
Presentation Layer	React components (Next.js)	Client-side rendering of interactive elements: the gallery lightbox, category filters, dynamic routing, and the inquiry form with CAPTCHA. Corresponds to the Boundary classes in Section 3.1.
Application / API Layer	Next.js API route handlers	Processes business logic, validates requests, and enforces authentication and role checks before touching the database. Corresponds to the Controller classes in Section 3.2.1.
Data / Persistence Layer	Managed PostgreSQL (Supabase)	Stores all relational data — packages, inquiries, gallery images, testimonials, site content, and announcements. Corresponds to the Model classes in Section 3.2.2 and the Data Dictionary in Section 3.5.

Table 2.1 – Three-Tier Architecture

2.2 Technology Stack

Layer	Technology	Rationale
Language	TypeScript	Gives compile-time type safety between the Prisma-generated database types and the application code, reducing schema/UI mismatches. Data arriving from outside the type system — form submissions, API payloads — is still checked at runtime with Zod, since types alone cannot validate external input.
Frontend framework	Next.js (App Router)	Supports Server-Side Rendering and Incremental Static Regeneration for the public site, improving load speed and crawlability for search engines — both genuine ranking factors, though actual search ranking also depends on content quality and competition, not the framework alone. The same codebase serves the authenticated admin dashboard.
UI library	React 18	Component-based UI with a large ecosystem; familiar to any future developer inheriting the codebase.
Styling	Tailwind CSS	Utility-first styling for rapid, consistent implementation of the design system established in the GUI mockups.
Backend / API	Next.js API Routes	Co-located with the frontend in the same Modular Monolith; no separate backend service to deploy or maintain.
Database	PostgreSQL (Supabase)	Relational integrity fits naturally relational data — packages, inquiries, and destination tags — better than a NoSQL store. Supabase adds managed hosting, backups, Auth, and Storage on top.
ORM	Prisma	Type-safe queries and schema migrations that generate the TypeScript types referenced above.

Authentication	Supabase Auth	Handles password hashing, session/JWT management, and role storage without custom auth code.
File storage	Supabase Storage	S3-compatible storage for package and gallery images, with bucket-level MIME-type and size-limit enforcement.
Email	Resend	Transactional email for inquiry notifications and admin invitations.
CAPTCHA	hCaptcha	Spam prevention on the public inquiry form.
Hosting	Vercel	Zero-config deployment for Next.js with automatic HTTPS; removes the need to manage Linux servers or scaling — appropriate for a small business with no dedicated IT staff.

Table 2.2 – Technology Stack

2.3 Design Alternative Considered

PHP with the Laravel framework (using Laravel Filament for the admin dashboard) was considered as an alternative stack. Laravel/Filament is a popular, capable choice for Malaysian SME content management systems and offers a strong all-in-one backend experience.

It was not selected for this system for two reasons specific to this project's constraints: it requires traditional server hosting and ongoing server management, which conflicts with the low-operational-overhead goal established for a small, family-run business without dedicated IT staff; and it does not provide the same out-of-the-box Server-Side Rendering and Incremental Static Regeneration that Next.js gives the public-facing pages, which matter for a content-heavy site that depends on organic search traffic for terms such as "Umrah packages Malaysia."

2.4 Pricing Data Design

Package pricing is a flat rate per person, set and updated by Admin users through the Manage Packages form (PackageFormPage) — there is no multi-currency or seasonal price-variation requirement, and no price-history table is needed since the system does not process live payments.

The `packages.price_per_pax` column is defined as `NUMERIC(10,2)` rather than `FLOAT` or `REAL`, which are unsuitable for currency values due to floating-point rounding error. `NUMERIC(10,2)` stores exact values up to 8 digits with 2 decimal places (e.g. 9800.00) — the standard approach for exact financial data in PostgreSQL. An alternative of storing the value as an `INTEGER` in the smallest currency unit (sen) was considered, since it simplifies future integration with payment gateways such as Stripe or Billplz, which typically expect integer minor units; it was not adopted for the current version because no payment gateway is in scope (per SRS-MTT-2026), and `NUMERIC(10,2)` keeps the value human-readable in the database without conversion logic scattered through the codebase. When a payment gateway is introduced in a future version, dedicated conversion logic can be added at the integration boundary rather than changing the storage type.

3. GENERAL ARCHITECTURE

3.1 Application Layer

The Application Layer contains the boundary classes — the pages and components a Visitor, Admin, or Master Admin directly interacts with — organised into the seven feature packages defined in the SRS. Each package below corresponds to one SRS module.

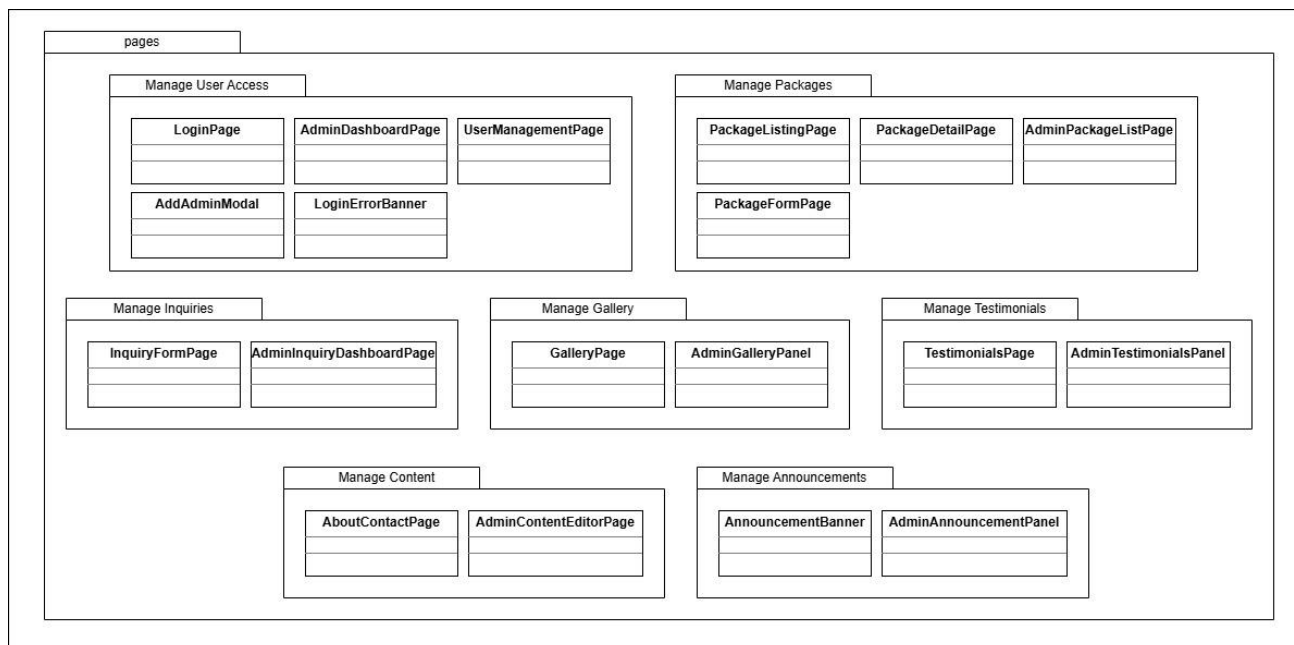


Figure 3.1 – Application Layer for the MTT System

3.1.1 MANAGE USER ACCESS [PKG-MTT-001]

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

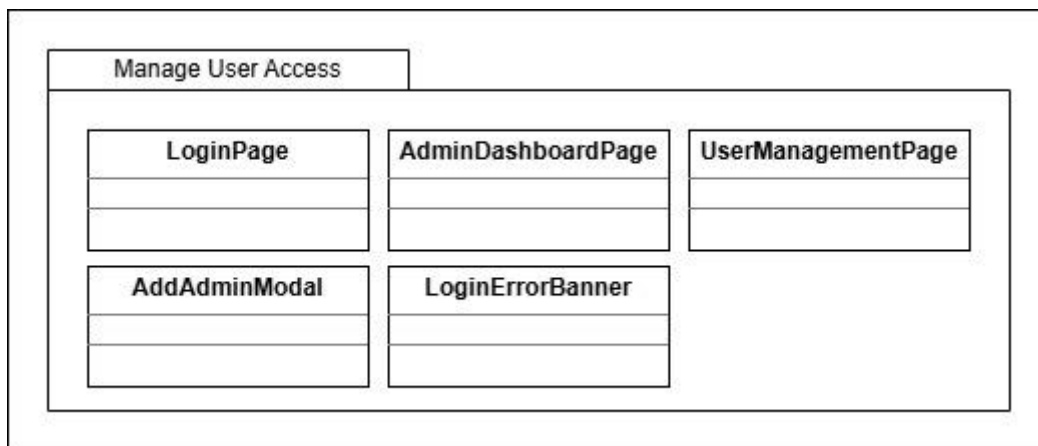


Figure 3.1.1 – MANAGE USER ACCESS Package Diagram

Class Name	Description
LoginPage [PKG-MTT-001-001]	Boundary class that securely captures Admin/Master Admin credentials and submits them to AuthController for authentication before granting dashboard access.
AdminDashboardPage [PKG-MTT-001-002]	Boundary class serving as the landing page after login. Displays summary statistics and navigation to all seven feature modules via the sidebar.
UserManagementPage	Boundary class, Master-Admin-only, listing all Admin accounts with options to add, deactivate, or delete staff access.

[PKG-MTT-001-003]	
AddAdminModal [PKG-MTT-001-004]	Boundary class providing a form for the Master Admin to invite a new staff member by name and email, triggering an invitation email.
LoginErrorBanner [PKG-MTT-001-005]	Reusable boundary component that displays invalid-credential or account-lockout messaging on LoginPage.

Table 3.1.1 – MANAGE USER ACCESS Package

3.1.2 MANAGE PACKAGES [PKG-MTT-002]

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

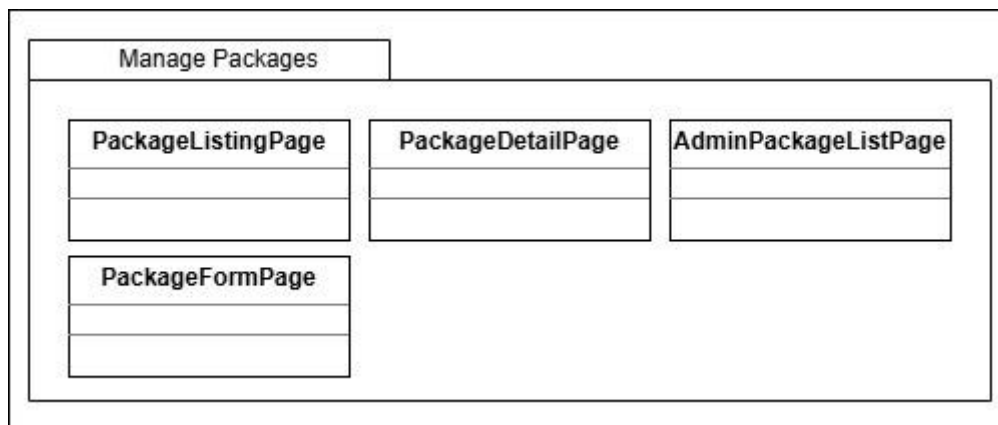


Figure 3.1.2 – MANAGE PACKAGES Package Diagram

Class Name	Description
PackageListingPage [PKG-MTT-002-001]	Boundary class, public-facing, displaying all published packages as cards, filterable by category (Umrah, Ziarah, Group Tour, Domestic).
PackageDetailPage [PKG-MTT-002-002]	Boundary class, public-facing, showing full package details including itinerary, pricing, departure dates, image gallery, and an Inquire Now action.
AdminPackageListPage [PKG-MTT-002-003]	Boundary class showing all packages, published and draft, with Edit and Delete actions for Admin users.
PackageFormPage [PKG-MTT-002-004]	Boundary class providing the form for Admin users to create or edit a package: details, itinerary, pricing, departure dates, and images.

Table 3.1.2 – MANAGE PACKAGES Package

3.1.3 MANAGE INQUIRIES [PKG-MTT-003]

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

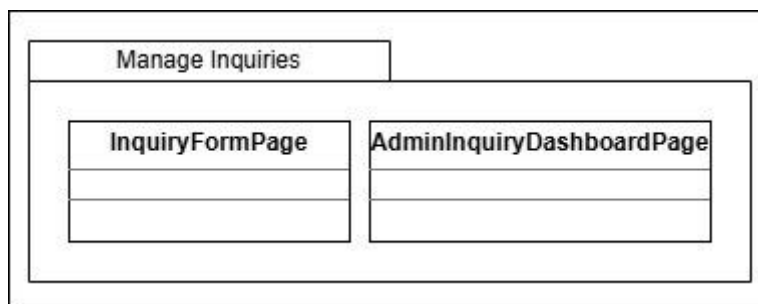


Figure 3.1.3 – MANAGE INQUIRIES Package Diagram

Class Name	Description
InquiryFormPage [PKG-MTT-003-001]	Boundary class, public-facing, capturing visitor contact information, package interest, message, and CAPTCHA response.
AdminInquiryDashboardPage [PKG-MTT-003-002]	Boundary class listing all inquiries with status badges, allowing Admin users to view details and update status.

Table 3.1.3 – MANAGE INQUIRIES Package

3.1.4 MANAGE GALLERY [PKG-MTT-004]

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

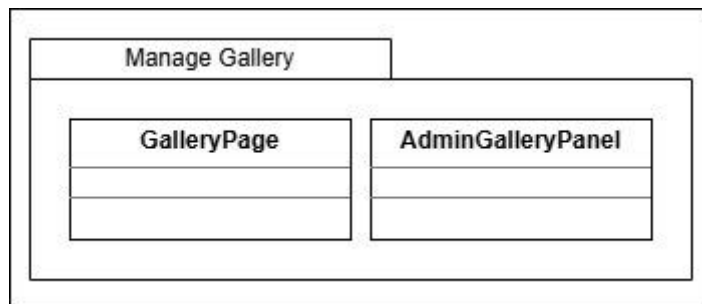


Figure 3.1.4 – MANAGE GALLERY Package Diagram

Class Name	Description
GalleryPage [PKG-MTT-004-001]	Boundary class, public-facing, displaying trip photographs in a responsive grid, filterable by destination tag.
AdminGalleryPanel [PKG-MTT-004-002]	Boundary class allowing Admin users to upload new photographs with a destination tag and delete existing ones.

Table 3.1.4 – MANAGE GALLERY Package

3.1.5 MANAGE TESTIMONIALS [PKG-MTT-005]

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

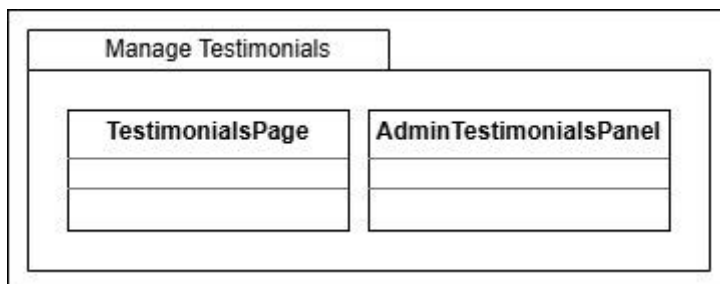


Figure 3.1.5 – MANAGE TESTIMONIALS Package Diagram

Class Name	Description
TestimonialsPage [PKG-MTT-005-001]	Boundary class, public-facing, displaying customer testimonials with star ratings on the homepage and a dedicated page.
AdminTestimonialsPanel [PKG-MTT-005-002]	Boundary class allowing Admin users to add, edit, and delete customer testimonials.

Table 3.1.5 – MANAGE TESTIMONIALS Package

3.1.6 MANAGE CONTENT PAGES [PKG-MTT-006]

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

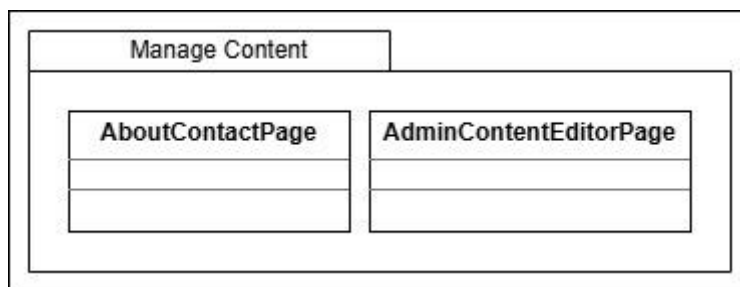


Figure 3.1.6 – MANAGE CONTENT PAGES Package Diagram

Class Name	Description
AboutContactPage [PKG-MTT-006-001]	Boundary class, public-facing, rendering the About Us and Contact pages from the current site_config values.
AdminContentEditorPage [PKG-MTT-006-002]	Boundary class allowing Admin users to edit About Us text, contact details, office hours, MOTAC licence info, and social links.

Table 3.1.6 – MANAGE CONTENT PAGES Package

3.1.7 MANAGE ANNOUNCEMENTS [PKG-MTT-007]

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

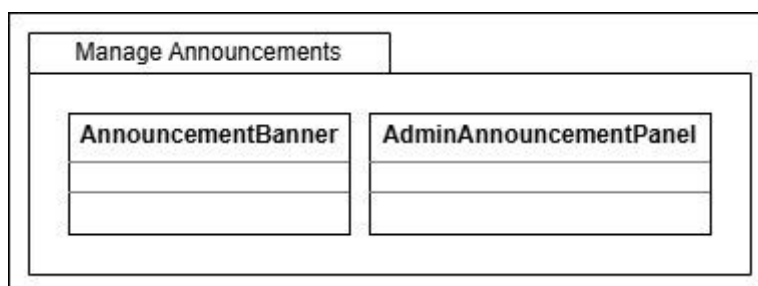


Figure 3.1.7 – MANAGE ANNOUNCEMENTS Package Diagram

Class Name	Description
AnnouncementBanner [PKG-MTT-007-001]	Boundary class, public-facing, conditionally displaying active, non-expired announcements at the top of the homepage.
AdminAnnouncementsPanel [PKG-MTT-007-002]	Boundary class allowing Admin users to create, edit, manually expire, or delete announcements.

Table 3.1.7 – MANAGE ANNOUNCEMENTS Package

3.2 Business Service Layer

The Business Service Layer sits between the Application Layer and the database. It contains Controller classes, which enforce business rules and validation before any data is written or returned, and Model classes, which mirror the database schema described in Section 3.5.



Figure 3.2 – Business Service Layer for the MTT System

3.2.1 Controllers [PKG-MTT-008]

Class Name	Description
AuthController [PKG-MTT-008-001]	Handles login, logout, session validation, admin invitation, and account deactivation/deletion, enforcing Master-Admin-only actions.
PackageController [PKG-MTT-008-002]	Handles package CRUD operations, publish/unpublish status, and category-filtered queries for the public listing and detail pages.
InquiryController [PKG-MTT-008-003]	Handles inquiry submission (including CAPTCHA verification), status updates, and deletion.
GalleryController [PKG-MTT-008-004]	Handles gallery image metadata creation and deletion, coordinating with SupabaseStorage for the underlying files.
TestimonialController [PKG-MTT-008-005]	Handles testimonial CRUD operations.

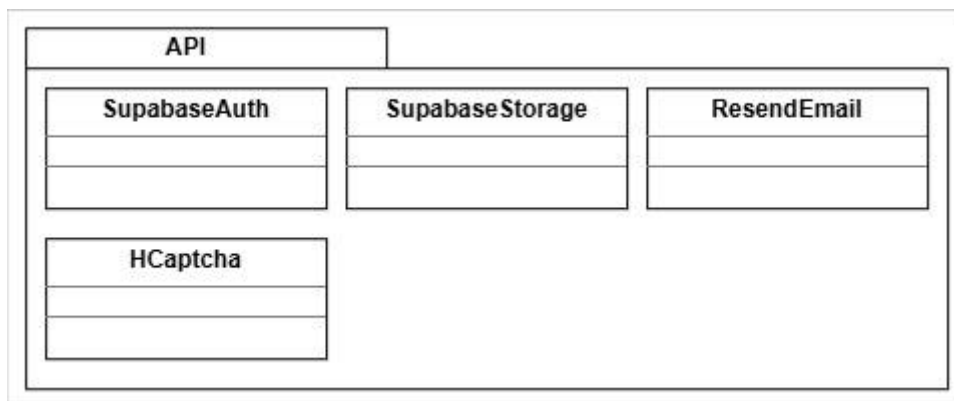
ContentController [PKG-MTT-008-006]	Handles reading and bulk-writing site_config key-value entries used by the public About/Contact pages.
AnnouncementController [PKG-MTT-008-007]	Handles announcement CRUD operations and expiry-aware queries for the public homepage banner.

Table 3.2.1 – Controller Classes**3.2.2 Models [PKG-MTT-009]**

Class Name	Description
User [PKG-MTT-009-001]	Stores Admin and Master Admin account data: role, active status, and metadata used for authentication and access control.
Package [PKG-MTT-009-002]	Stores travel package data: title, category, description, itinerary, pricing, status, and departure dates.
PackagelImage [PKG-MTT-009-003]	Stores image metadata associated with a package, including storage path, primary flag, and display order.
Inquiry [PKG-MTT-009-004]	Stores visitor inquiry submissions: contact details, package interest, message, and status.
GalleryImage [PKG-MTT-009-005]	Stores gallery photo metadata including storage path, destination tag, and uploader.
Testimonial [PKG-MTT-009-006]	Stores customer testimonial data: name, trip, review text, and star rating.
SiteConfig [PKG-MTT-009-007]	Key-value store for editable site content such as About text, contact details, and social links.
Announcement [PKG-MTT-009-008]	Stores announcement data: title, body, status, and optional expiry timestamp.
AuditLog [PKG-MTT-009-009]	Stores a record of Admin actions for accountability, including the user, action, target, and a before/after snapshot.

Table 3.2.2 – Model Classes**3.3 Middleware Layer [PKG-MTT-010]**

The Middleware Layer contains the external, third-party services the MTT System depends on. Controllers call these services rather than implementing authentication, storage, email, or CAPTCHA verification directly.

**Figure 3.3 – Middleware Layer for the MTT System**

Class Name	Description
SupabaseAuth [PKG-MTT-010-001]	Handles authentication, session/JWT management, password hashing, and role-based session data for the MTT System.
SupabaseStorage [PKG-MTT-010-002]	Handles file storage for package and gallery images, including bucket-level MIME-type and size-limit enforcement.
ResendEmail [PKG-MTT-010-003]	Handles transactional email delivery: inquiry notifications to Admin and account invitation emails to new staff.
HCaptcha [PKG-MTT-010-004]	Verifies CAPTCHA tokens submitted with public inquiry forms to prevent automated spam submissions.

Table 3.3 – Middleware Classes

3.4 Package Module

The diagram below shows how the ten packages relate: the seven Application Layer packages each depend on one or more Controllers, which in turn depend on Models and the Middleware Layer.

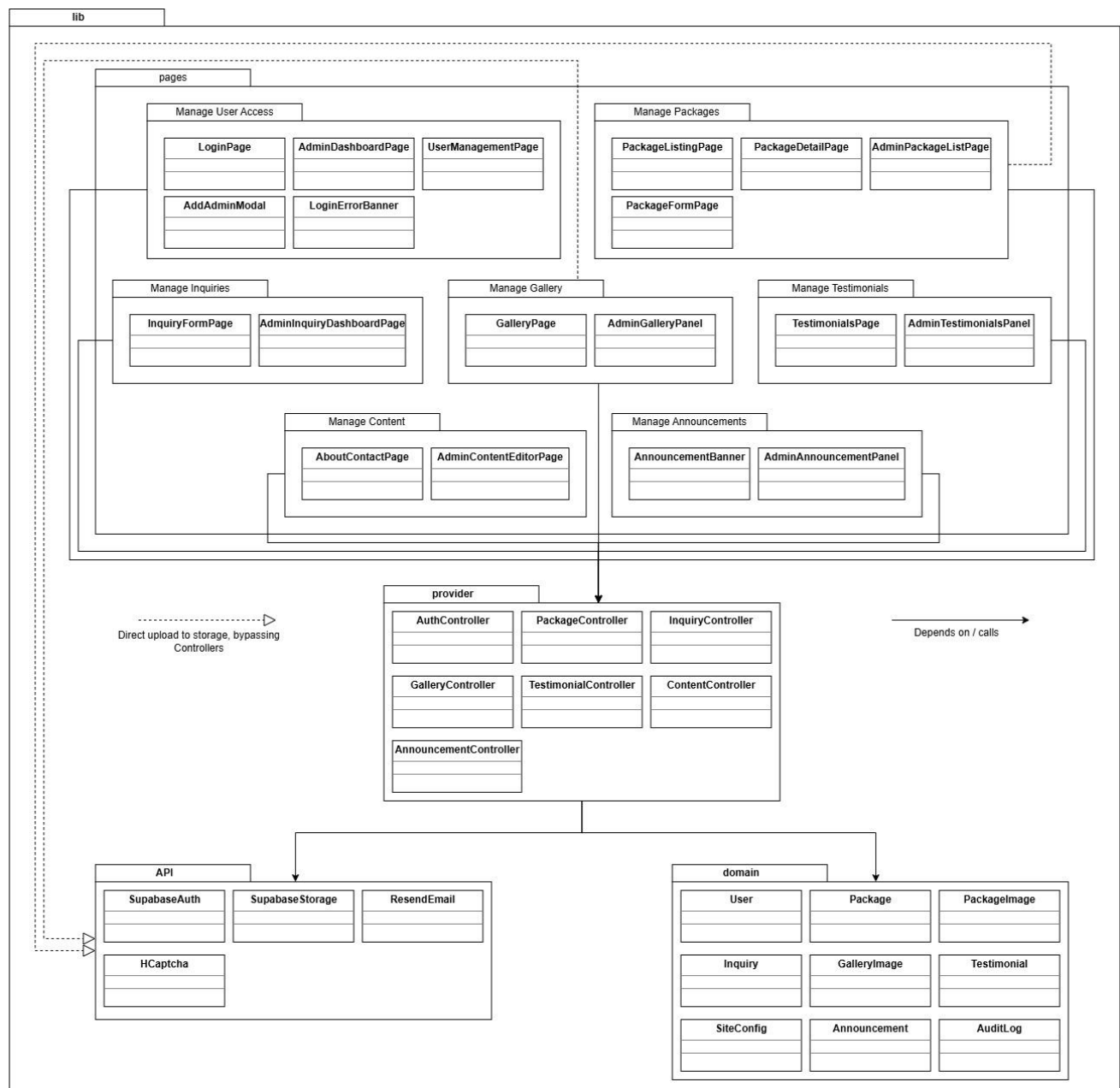


Figure 3.4 – Package Module for the MTT System

3.5 Data Dictionary

The Data Dictionary defines every table in the PostgreSQL database, one table per Model class defined in Section 3.2.2.

3.5 Table users

Field Name	Description	Data Type	Constraint
id	Primary key, maps to Supabase Auth user ID	UUID	PK
email	Staff Gmail or company email address	VARCHAR	UNIQUE, NOT NULL
name	Display name of the staff member	VARCHAR	NOT NULL
role	MASTER_ADMIN or ADMIN	ENUM	NOT NULL
is_active	FALSE = deactivated, cannot log in	BOOLEAN	NOT NULL, default TRUE
failed_attempts	Consecutive failed login attempts	INTEGER	default 0
locked_until	Lockout expiry after 5 failed attempts	TIMESTAMPTZ	
created_at	Account creation timestamp	TIMESTAMPTZ	NOT NULL

Table 3.5.1 – Table users

3.5 Table packages

Field Name	Description	Data Type	Constraint
id	Primary key	UUID	PK
title	Package name	VARCHAR	NOT NULL
category	UMRAH, ZIARAH, GROUP_TOUR, or DOMESTIC	ENUM	NOT NULL
description	Full package description	TEXT	NOT NULL
itinerary	Day-by-day itinerary content	TEXT	NOT NULL
price_per_pax	Base price per person in MYR	DECIMAL	NOT NULL
status	PUBLISHED, DRAFT, COMING_SOON, or FULL	ENUM	NOT NULL, default DRAFT
departure_dates	Array of departure date strings	TEXT[]	
created_by	Admin who created the package	UUID	FK → users.id
deleted_at	Soft-delete timestamp; NULL = active	TIMESTAMPTZ	

Table 3.5.2 – Table packages

3.5 Table package_images

Field Name	Description	Data Type	Constraint
id	Primary key	UUID	PK
package_id	Parent package	UUID	FK → packages.id
storage_path	Supabase Storage object path	VARCHAR	NOT NULL
url	Public URL of the image	TEXT	NOT NULL
is_primary	TRUE = shown as the card thumbnail	BOOLEAN	default FALSE
sort_order	Display order within the gallery	INTEGER	default 0

Table 3.5.3 – Table package_images**3.5 Table inquiries**

Field Name	Description	Data Type	Constraint
id	Primary key	UUID	PK
full_name	Visitor full name	VARCHAR	NOT NULL
phone	Visitor phone number	VARCHAR	NOT NULL
email	Visitor email address	VARCHAR	NOT NULL
package_interest	Package name the visitor is interested in	VARCHAR	
message	Visitor message body	TEXT	NOT NULL
status	NEW, IN_PROGRESS, or RESOLVED	ENUM	NOT NULL, default NEW
created_at	Submission timestamp	TIMESTAMPTZ	NOT NULL

Table 3.5.4 – Table inquiries**3.5 Table gallery_images**

Field Name	Description	Data Type	Constraint
id	Primary key	UUID	PK
storage_path	Supabase Storage object path	VARCHAR	NOT NULL
url	Public URL of the image	TEXT	NOT NULL
tag	Destination or package label	VARCHAR	NOT NULL
uploaded_by	Admin who uploaded the image	UUID	FK → users.id

Table 3.5.5 – Table gallery_images**3.5 Table testimonials**

Field Name	Description	Data Type	Constraint
id	Primary key	UUID	PK
customer_name	Customer display name	VARCHAR	NOT NULL
trip_name	Trip or package the customer took	VARCHAR	NOT NULL
review_text	Full review content	TEXT	NOT NULL
star_rating	Rating from 1 to 5	SMALLINT	NOT NULL
trip_date	Approximate date of the trip	DATE	

Table 3.5.6 – Table testimonials**3.5 Table site_config**

Field Name	Description	Data Type	Constraint
key	Content identifier, e.g. about_text, phone	VARCHAR	PK

value	Content value: plain text, HTML, or URL	TEXT	NOT NULL
updated_by	Admin who last updated this entry	UUID	FK → users.id
updated_at	Last update timestamp	TIMESTAMPTZ	NOT NULL

*Table 3.5.7 – Table site_config***3.5 Table announcements**

Field Name	Description	Data Type	Constraint
id	Primary key	UUID	PK
title	Announcement headline	VARCHAR	NOT NULL
body	Full announcement text	TEXT	NOT NULL
status	ACTIVE or EXPIRED	ENUM	NOT NULL, default ACTIVE
expires_at	Auto-expire datetime; NULL = never	TIMESTAMPTZ	
created_by	Admin who created the announcement	UUID	FK → users.id

*Table 3.5.8 – Table announcements***3.5 Table audit_log**

Field Name	Description	Data Type	Constraint
id	Primary key	UUID	PK
user_id	Admin who performed the action	UUID	FK → users.id
action	Action label, e.g. CREATE_PACKAGE	VARCHAR	NOT NULL
target	Entity affected, e.g. packages:uuid	VARCHAR	NOT NULL
detail	Before/after snapshot of changed data	JSONB	
created_at	Action timestamp	TIMESTAMPTZ	NOT NULL

Table 3.5.9 – Table audit_log

4. DETAIL DESIGN

This section provides a full Attributes / Methods / Algorithm breakdown for the fourteen classes most valuable as an implementation reference: the primary boundary class from each of the seven feature modules, and all seven Controller classes, where the system's real business logic lives. The remaining boundary classes listed in Section 3.1 follow the same CRUD and validation patterns shown here and are not repeated in full.

Mirza Naufal Beg bin Mirza Mohammad Raziq Beg

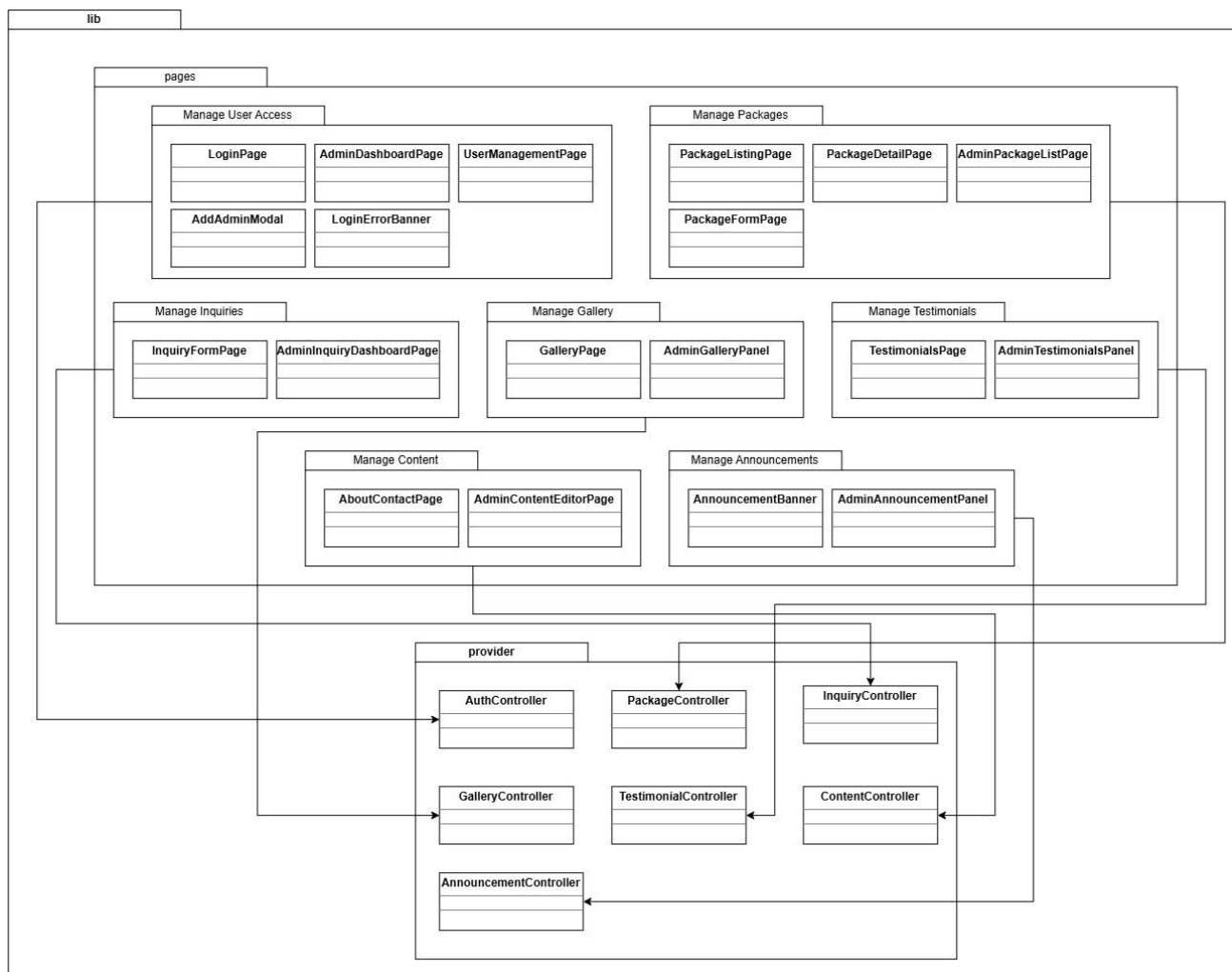


Figure 4.1 – Detail Design Overview for the MTT System

4.1 Boundary Class Detail Design

4.1.1 Class LoginPage [PKG-MTT-001-001]

Item	Description	
Class Type	Boundary class	
Responsibility	Securely captures Admin/Master Admin credentials and submits them to AuthController for authentication, displaying lockout messaging after repeated failures.	
Attributes	Attribute	Type
	email	VARCHAR
	password	VARCHAR
	errorMessage	VARCHAR

	failedAttempts	INTEGER
Methods	Method Name	Description
	displayLoginForm()	Renders the login form UI.
	captureCredentials()	Temporarily holds the email and password input.
	submitLogin()	Sends credentials to AuthController for verification.
	handleLoginError()	Displays an invalid-credential or lockout message.
	redirectToDashboard()	Navigates to AdminDashboardPage on success.
Algorithm	START 1. System executes displayLoginForm() to render the login page. 2. Admin enters email and password; system executes captureCredentials(). 3. Admin clicks Sign In, triggering submitLogin(). 4. AuthController.signInWithPassword() verifies the credentials. 5. IF credentials are valid: AuthController creates a session and returns role and is_active; system executes redirectToDashboard(). 6. ELSE: system executes handleLoginError() to display "Invalid email or password"; failedAttempts is incremented. 7. IF failedAttempts reaches 5: AuthController sets locked_until to 15 minutes from now and handleLoginError() displays the lockout message instead. END	

Table 4.1.1 – Class LoginPage

4.1.2 Class PackageFormPage [PKG-MTT-002-004]

Item	Description	
Class Type	Boundary class	
Responsibility	Provides the form for Admin users to create or edit a package, capturing details, itinerary, pricing, departure dates, and images before submitting to PackageController.	
Attributes	Attribute	Type
	title	VARCHAR
	category	VARCHAR
	description	TEXT
	pricePerPax	DECIMAL
	availability	VARCHAR
	departureDates	ARRAY
	images	ARRAY
Methods	Method Name	Description
	displayForm()	Renders the add/edit package form, pre-filled when editing.
	captureFields()	Temporarily holds form input as the Admin types.
	validateFields()	Checks required fields client-side before submission.
	uploadImages()	Sends selected image files to SupabaseStorage.
	submitPackage()	Sends the structured package data to PackageController.
	handleValidationError()	Highlights missing or invalid fields on failure.

Algorithm	START 1. Admin clicks Add New Package or Edit; system executes displayForm(). 2. Admin fills in title, category, description, itinerary, price, and departure dates; system executes captureFields(). 3. Admin selects image files; system executes uploadImages(), which uploads to SupabaseStorage and returns image URLs. 4. Admin clicks Save as Draft or Publish, triggering submitPackage(). 5. System executes validateFields(). 6. IF validation fails: system executes handleValidationError() and returns to step 2. 7. ELSE: PackageController.createPackage() or updatePackage() writes the record and image metadata to the database. 8. IF status is Published: the package becomes immediately visible on PackageListingPage. END
------------------	---

Table 4.1.2 – Class PackageFormPage

4.1.3 Class InquiryFormPage [PKG-MTT-003-001]

Item	Description	
Class Type	Boundary class	
Responsibility	Captures visitor contact information, package interest, message, and CAPTCHA response, and submits the inquiry to InquiryController.	
Attributes	Attribute	Type
	fullName	VARCHAR
	phone	VARCHAR
	email	VARCHAR
	packageInterest	VARCHAR
	message	TEXT
	captchaToken	VARCHAR
Methods	Method Name	Description
	displayForm()	Renders the inquiry form, pre-filling packageInterest if arriving from a package page.
	captureFields()	Temporarily holds form input.
	solveCaptcha()	Captures the hCaptcha widget token once solved.
	submitInquiry()	Sends the form data and captchaToken to InquiryController.
	displaySuccessMessage()	Confirms submission to the visitor.
Algorithm	START 1. Visitor navigates to the Inquiry page; system executes displayForm(). 2. Visitor fills in the required fields; system executes captureFields(). 3. Visitor solves the CAPTCHA; system executes solveCaptcha(). 4. Visitor clicks Submit, triggering submitInquiry(). 5. InquiryController verifies the captchaToken with the HCaptcha middleware. 6. IF the CAPTCHA is invalid, or a required field is empty: system displays a validation error and returns to step 2.	

	7. ELSE: InquiryController.createInquiry() inserts the record into the database with status "New". 8. InquiryController calls ResendEmail.sendInquiryNotification() to alert the Admin. IF this fails, the failure is logged internally without affecting the saved inquiry. 9. System executes displaySuccessMessage(). END
--	--

Table 4.1.3 – Class InquiryFormPage**4.1.4 Class AdminGalleryPanel [PKG-MTT-004-002]**

Item	Description	
Class Type	Boundary class	
Responsibility	Allows Admin users to upload new gallery photographs with a destination tag and delete existing ones.	
Attributes	Attribute	Type
	selectedFiles	ARRAY
	tag	VARCHAR
	images	ARRAY
Methods	Method Name	Description
	displayPanel()	Renders the gallery grid with Upload and Delete controls.
	selectFiles()	Captures the image files chosen from the Admin's device.
	validateFileType()	Checks file format and size client-side before upload.
	uploadToStorage()	Sends validated files to SupabaseStorage.
	saveImageMetadata()	Sends the tag and storage URLs to GalleryController.
	deleteImage()	Removes an image from storage and its database record.
Algorithm	START 1. Admin clicks Upload Photos; system executes displayPanel(). 2. Admin executes selectFiles() and assigns a destination tag. 3. System executes validateFileType() on each selected file. 4. IF a file is invalid: system displays a file error and returns to step 2. 5. ELSE: system executes uploadToStorage(), then executes saveImageMetadata() via GalleryController.createImage(). 6. Uploaded images appear immediately in the public GalleryPage. 7. On delete: Admin executes deleteImage(); the file is removed from SupabaseStorage and the record is removed from the database. END	

Table 4.1.4 – Class AdminGalleryPanel**4.1.5 Class AdminTestimonialsPanel [PKG-MTT-005-002]**

Item	Description
Class Type	Boundary class
Responsibility	Allows Admin users to add, edit, and delete customer testimonials shown on the public site.

Attributes	Attribute	Type
	customerName	VARCHAR
	tripName	VARCHAR
	reviewText	TEXT
	starRating	INTEGER
	tripDate	DATE
Methods	Method Name	Description
	displayPanel()	Renders the list of testimonials with Add, Edit, and Delete controls.
	captureFields()	Temporarily holds form input for a new or edited testimonial.
	validateFields()	Checks that customer name, trip name, and review text are present.
	submitTestimonial()	Sends the testimonial data to TestimonialController.
	deleteTestimonial()	Removes a testimonial record.
Algorithm	START 1. Admin clicks Add Testimonial; system executes displayPanel(). 2. Admin executes captureFields(). 3. Admin clicks Save, triggering submitTestimonial(). 4. System executes validateFields(). 5. IF invalid: system highlights the missing fields and returns to step 2. 6. ELSE: TestimonialController.createTestimonial() inserts the record; it appears immediately on the public TestimonialsPage. END	

Table 4.1.5 – Class AdminTestimonialsPanel

4.1.6 Class AdminContentEditorPage [PKG-MTT-006-002]

Item	Description	
Class Type	Boundary class	
Responsibility	Allows Admin users to edit the About Us text, contact details, office hours, MOTAC licence information, and social links stored in site_config.	
Attributes	Attribute	Type
	aboutText	TEXT
	contactAddress	VARCHAR
	contactPhone	VARCHAR
	officeHours	VARCHAR
	motacLicense	VARCHAR
	socialLinks	ARRAY
Methods	Method Name	Description
	displayEditor()	Renders the content form, pre-filled from site_config.
	captureFields()	Temporarily holds edited field values.

	validateFormat()	Checks social URL and phone number formatting.
	submitChanges()	Sends the updated key-value pairs to ContentController.
Algorithm	START - Admin navigates to Manage Content; system executes displayEditor(). ContentController.getContent() populates all fields. - Admin edits one or more sections; system executes captureFields(). - Admin clicks Save Changes, triggering submitChanges(). - System executes validateFormat(). - IF invalid: system highlights the invalid field and returns to step 2. - ELSE: ContentController.updateContent() writes the changes to site_config in a single bulk update. - Public pages reflect the updated content immediately. END	

Table 4.1.6 – Class AdminContentEditorPage

4.1.7 Class AdminAnnouncementsPanel [PKG-MTT-007-002]

Item	Description	
Class Type	Boundary class	
Responsibility	Allows Admin users to create, edit, manually expire, or delete time-sensitive announcements.	
Attributes	Attribute	Type
	title	VARCHAR
	body	TEXT
	expiresAt	DATE
	status	VARCHAR
Methods	Method Name	Description
	displayPanel()	Renders the list of announcements with Add, Edit, Expire, and Delete controls.
	captureFields()	Temporarily holds form input.
	submitAnnouncement()	Sends the announcement data to AnnouncementController.
	expireAnnouncement()	Marks an announcement's status as Expired.
	deleteAnnouncement()	Removes an announcement record.
Algorithm	START - Admin clicks Add Announcement; system executes displayPanel(). - Admin executes captureFields(), optionally setting expiresAt. - Admin clicks Publish, triggering submitAnnouncement(). - IF title or body is empty: system highlights the missing fields and returns to step 2. - ELSE: AnnouncementController.createAnnouncement() inserts the record with status "Active". - The announcement appears immediately on the public AnnouncementBanner if not expired. - On expire or delete: Admin executes expireAnnouncement() or deleteAnnouncement(); the record is updated or removed and no longer displayed publicly.	

	END
--	-----

Table 4.1.7 – Class AdminAnnouncementsPanel

4.2 Controller Class Detail Design

4.2.1 Class AuthController [PKG-MTT-008-001]

Item	Description	
Class Type	Control class	
Responsibility	Handles login, logout, session validation, admin invitation, and account deactivation/deletion, enforcing that only the Master Admin can manage staff accounts.	
Attributes	Attribute	Type
	userId	UUID
	email	VARCHAR
	role	VARCHAR
	isActive	BOOLEAN
	sessionToken	VARCHAR
Methods	Method Name	Description
	signInWithPassword()	Verifies credentials via SupabaseAuth and creates a session.
	signOut()	Invalidates the current session.
	inviteAdmin()	Creates a new Auth user and sends an invitation email.
	deactivateAdmin()	Sets is_active to false and disables the Auth session.
	deleteAdmin()	Removes the Admin account record.
	verifyRole()	Checks the requesting user holds the required role before allowing an action.
Algorithm	START 1. Receives a login request with email and password from LoginPage. 2. Calls SupabaseAuth.signInWithPassword(). 3. IF invalid: increments failed_attempts on the users record; IF failed_attempts reaches 5, sets locked_until to 15 minutes from now. 4. IF valid: retrieves role and is_active from the users table and returns a session to LoginPage. 5. On an invite request: executes verifyRole() to confirm the caller is Master Admin, then calls SupabaseAuth.inviteUserByEmail() and ResendEmail.sendInvitationEmail(). 6. On a deactivate or delete request: executes verifyRole(), then updates or removes the corresponding users record and writes an audit_log entry. END	

Table 4.2.1 – Class AuthController

4.2.2 Class PackageController [PKG-MTT-008-002]

Item	Description	
Class Type	Control class	
Responsibility	Handles package CRUD operations, publish/unpublish status, and category-filtered queries for the public listing and detail pages.	
Attributes	Attribute	Type
	packageId	UUID
	title	VARCHAR

	category	VARCHAR
	status	VARCHAR
	pricePerPax	DECIMAL
Methods	Method Name	Description
	listPublishedPackages()	Returns all packages with status Published, optionally filtered by category.
	getPackageById()	Returns full package details including images.
	createPackage()	Validates and inserts a new package record and image metadata.
	updatePackage()	Validates and updates an existing package record.
	deletePackage()	Soft-deletes a package by setting deleted_at.
Algorithm	START 1. Receives a create or update request from PackageFormPage. 2. Validates required fields: title, category, and at least one image. 3. IF invalid: returns a 400 response with field-level errors. 4. ELSE: writes the package record to the packages table and associated rows to package_images. 5. On a delete request: sets deleted_at on the target record rather than removing it, preserving it for audit. 6. On public list or detail requests: queries only rows where status = Published and deleted_at IS NULL. END	

Table 4.2.2 – Class PackageController

4.2.3 Class InquiryController [PKG-MTT-008-003]

Item	Description	
Class Type	Control class	
Responsibility	Handles inquiry submission, including CAPTCHA verification and email notification, plus status updates and deletion by Admin.	
Attributes	Attribute	Type
	inquiryId	UUID
	fullName	VARCHAR
	status	VARCHAR
Methods	Method Name	Description
	createInquiry()	Verifies CAPTCHA, validates fields, and inserts a new inquiry with status New.
	listInquiries()	Returns all inquiries sorted by date, optionally filtered by status.
	updateInquiryStatus()	Updates an inquiry's status to In Progress or Resolved.
	deleteInquiry()	Removes an inquiry record.
Algorithm	START 1. Receives a submission from InquiryFormPage. 2. Calls the HCaptcha middleware to verify the captchaToken.	

	3. IF the CAPTCHA is invalid: returns an error to the visitor. 4. Validates required fields: name, phone, email, and message. 5. IF invalid: returns field-level errors. 6. ELSE: inserts the record into the inquiries table with status "New". 7. Calls ResendEmail.sendInquiryNotification(). IF this fails, logs the failure internally without affecting the saved inquiry or the visitor's success response. 8. On a status update request from AdminInquiryDashboardPage: updates the status field and updated_at timestamp. END
--	---

Table 4.2.3 – Class InquiryController

4.2.4 Class GalleryController [PKG-MTT-008-004]

Item	Description	
Class Type	Control class	
Responsibility	Handles gallery image metadata creation and deletion, coordinating with SupabaseStorage for the underlying files.	
Attributes	Attribute	Type
	imageId	UUID
	tag	VARCHAR
	storagePath	VARCHAR
Methods	Method Name	Description
	listImages()	Returns all gallery images, optionally filtered by tag.
	createImage()	Inserts image metadata after a successful SupabaseStorage upload.
	deleteImage()	Removes an image from SupabaseStorage and its database record.
Algorithm	START 1. Receives an upload request from AdminGalleryPanel after files are uploaded to SupabaseStorage. 2. Inserts a record into gallery_images with the tag and storage URL. 3. On a public list request: queries gallery_images, optionally filtered by tag. 4. On a delete request: calls SupabaseStorage to remove the file, then deletes the database record. END	

Table 4.2.4 – Class GalleryController

4.2.5 Class TestimonialController [PKG-MTT-008-005]

Item	Description	
Class Type	Control class	
Responsibility	Handles testimonial CRUD operations for Admin users and read access for the public site.	
Attributes	Attribute	Type
	testimonialId	UUID

	customerName	VARCHAR
	starRating	INTEGER
Methods	Method Name	Description
	listTestimonials()	Returns all testimonials sorted by date.
	createTestimonial()	Validates and inserts a new testimonial.
	updateTestimonial()	Updates an existing testimonial.
	deleteTestimonial()	Removes a testimonial record.
Algorithm	START 1. Receives a create or update request from AdminTestimonialsPanel. 2. Validates required fields: customer name, trip name, and review text. 3. IF invalid: returns field-level errors. 4. ELSE: writes the record to the testimonials table. 5. On a public request: returns all testimonials ordered by created_at descending. END	

Table 4.2.5 – Class TestimonialController

4.2.6 Class ContentController [PKG-MTT-008-006]

Item	Description	
Class Type	Control class	
Responsibility	Handles reading and bulk-writing site_config key-value entries used to render the public About and Contact pages.	
Attributes	Attribute	Type
	key	VARCHAR
	value	TEXT
Methods	Method Name	Description
	getContent()	Returns all site_config key-value entries.
	updateContent()	Validates and writes one or more key-value entries in a single request.
Algorithm	START 1. Receives an update request from AdminContentEditorPage containing one or more key-value pairs. 2. Validates format-sensitive fields: social URLs and phone number. 3. IF invalid: returns field-level errors. 4. ELSE: writes each key-value pair to the site_config table in a single bulk update. 5. On a public request: returns all site_config entries for rendering the About and Contact pages. END	

Table 4.2.6 – Class ContentController

4.2.7 Class AnnouncementController [PKG-MTT-008-007]

Item	Description
-------------	--------------------

Class Type	Control class	
Responsibility	Handles announcement CRUD operations and expiry-aware queries for the public homepage banner.	
Attributes	Attribute	Type
	announcementId	UUID
	title	VARCHAR
	status	VARCHAR
	expiresAt	TIMESTAMP
Methods	Method Name	Description
	listActiveAnnouncements()	Returns announcements with status Active and expiresAt in the future or null.
	createAnnouncement()	Validates and inserts a new announcement with status Active.
	updateAnnouncement()	Updates an existing announcement.
	expireAnnouncement()	Sets an announcement's status to Expired.
	deleteAnnouncement()	Removes an announcement record.
Algorithm	START 1. Receives a create request from AdminAnnouncementsPanel. 2. Validates required fields: title and body. 3. IF invalid: returns field-level errors. 4. ELSE: inserts the record with status "Active". 5. On a public homepage request: queries announcements where status = Active AND (expiresAt IS NULL OR expiresAt > now()). 6. On an expire or delete request: updates status to "Expired" or removes the record. END	

Table 4.2.7 – Class AnnouncementController

5. REQUIREMENTS TRACEABILITY

The table below maps every requirement defined in SRS-MTT-2026 to the package and class(es) in this design responsible for satisfying it.

Requirement ID	Package	Description	Responsible Class
REQ-MTT-001-001	PKG-MTT-001	Admin/Master Admin can log in with email and password	LoginPage, AuthController
REQ-MTT-001-002	PKG-MTT-001	System invalidates session on logout	AdminDashboardPage, AuthController
REQ-MTT-001-003	PKG-MTT-001	Master Admin can create a new Admin account	AddAdminModal, AuthController
REQ-MTT-001-004	PKG-MTT-001	System sends invitation email with setup link	AuthController, ResendEmail
REQ-MTT-001-005	PKG-MTT-001	Master Admin can deactivate or delete an Admin account	UserManagementPage, AuthController
REQ-MTT-001-006	PKG-MTT-001	Account locks for 15 minutes after 5 failed attempts	LoginErrorBanner, AuthController
REQ-MTT-002-001	PKG-MTT-002	Admin can create a package with required fields and an image	PackageFormPage, PackageController
REQ-MTT-002-002	PKG-MTT-002	System saves package to database on valid submission	PackageController, Package
REQ-MTT-002-003	PKG-MTT-002	Published packages are immediately visible publicly	PackageListingPage, PackageController
REQ-MTT-002-004	PKG-MTT-002	Visitor can browse packages filtered by category	PackageListingPage, PackageController
REQ-MTT-002-005	PKG-MTT-002	Visitor can view full package detail page	PackageDetailPage, PackageController
REQ-MTT-002-006	PKG-MTT-002	Admin can edit an existing package	PackageFormPage, PackageController
REQ-MTT-002-007	PKG-MTT-002	Admin can delete a package after confirmation	AdminPackageListPage, PackageController
REQ-MTT-003-001	PKG-MTT-003	Visitor can navigate to the inquiry form	InquiryFormPage
REQ-MTT-003-002	PKG-MTT-003	Inquiry form captures name, phone, email, package, message	InquiryFormPage, Inquiry
REQ-MTT-003-003	PKG-MTT-003	CAPTCHA must be solved before submission	InquiryFormPage, HCaptcha
REQ-MTT-003-004	PKG-MTT-003	System saves inquiry with status New	InquiryController, Inquiry
REQ-MTT-003-005	PKG-MTT-003	System sends email notification to Admin	InquiryController, ResendEmail
REQ-MTT-003-006	PKG-MTT-003	System displays success message to visitor	InquiryFormPage
REQ-MTT-003-007	PKG-MTT-003	Admin can view all inquiries with status badges	AdminInquiryDashboardPage, InquiryController
REQ-MTT-003-008	PKG-MTT-003	Admin can update inquiry status	AdminInquiryDashboardPage, InquiryController
REQ-MTT-003-009	PKG-MTT-003	Admin can delete an inquiry	AdminInquiryDashboardPage, InquiryController

REQ-MTT-004-001	PKG-MTT-004	Admin can upload one or more images	AdminGalleryPanel, SupabaseStorage
REQ-MTT-004-002	PKG-MTT-004	Admin assigns a destination tag to uploaded images	AdminGalleryPanel, GalleryController
REQ-MTT-004-003	PKG-MTT-004	System uploads images and saves metadata	GalleryController, GalleryImage
REQ-MTT-004-004	PKG-MTT-004	Uploaded images appear immediately in the gallery	GalleryPage, GalleryController
REQ-MTT-004-005	PKG-MTT-004	Visitor can browse gallery in a responsive grid	GalleryPage
REQ-MTT-004-006	PKG-MTT-004	Visitor can filter gallery images by destination	GalleryPage, GalleryController
REQ-MTT-004-007	PKG-MTT-004	Admin can delete a gallery image	AdminGalleryPanel, GalleryController
REQ-MTT-005-001	PKG-MTT-005	Admin can add a testimonial with rating	AdminTestimonialsPanel, TestimonialController
REQ-MTT-005-002	PKG-MTT-005	Testimonial is visible immediately after save	TestimonialsPage, TestimonialController
REQ-MTT-005-003	PKG-MTT-005	Visitor can view all testimonials, newest first	TestimonialsPage, TestimonialController
REQ-MTT-005-004	PKG-MTT-005	Admin can edit an existing testimonial	AdminTestimonialsPanel, TestimonialController
REQ-MTT-005-005	PKG-MTT-005	Admin can delete a testimonial	AdminTestimonialsPanel, TestimonialController
REQ-MTT-006-001	PKG-MTT-006	Admin content editor is pre-populated with current content	AdminContentEditorPage, ContentController
REQ-MTT-006-002	PKG-MTT-006	Admin can edit the About Us story	AdminContentEditorPage, SiteConfig
REQ-MTT-006-003	PKG-MTT-006	Admin can edit contact details	AdminContentEditorPage, SiteConfig
REQ-MTT-006-004	PKG-MTT-006	Admin can edit office hours	AdminContentEditorPage, SiteConfig
REQ-MTT-006-005	PKG-MTT-006	Admin can edit MOTAC licence number	AdminContentEditorPage, SiteConfig
REQ-MTT-006-006	PKG-MTT-006	Admin can edit social media links	AdminContentEditorPage, SiteConfig
REQ-MTT-006-007	PKG-MTT-006	Public pages reflect updated content immediately	AboutContactPage, ContentController
REQ-MTT-007-001	PKG-MTT-007	Admin can create an announcement with optional expiry	AdminAnnouncementsPanel, AnnouncementController
REQ-MTT-007-002	PKG-MTT-007	System saves announcement with status Active	AnnouncementController, Announcement
REQ-MTT-007-003	PKG-MTT-007	Active announcement displays on homepage immediately	AnnouncementBanner, AnnouncementController
REQ-MTT-007-004	PKG-MTT-007	System hides expired announcements automatically	AnnouncementBanner, AnnouncementController
REQ-MTT-007-005	PKG-MTT-007	Admin can edit an existing announcement	AdminAnnouncementsPanel, AnnouncementController

REQ-MTT-007-006	PKG-MTT-007	Admin can manually expire or delete an announcement	AdminAnnouncementsPanel, AnnouncementController
-----------------	-------------	---	---

Table 5.0 – Requirements Traceability for the MTT System